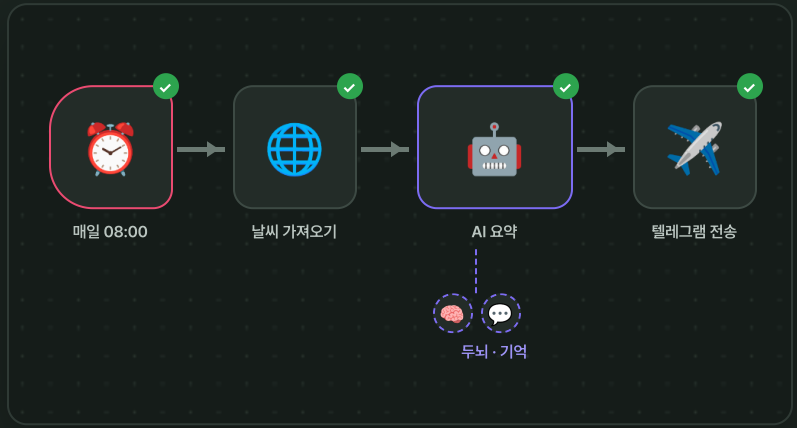
강의 자료 · 쉬운 요약

n8n,
진짜 쉽게

코드 없이 블록을 이어 만드는 자동화 · 단계별 용어 풀이와 클릭 순서

Trigger Node AI Agent



목차

들어가며	이 자료 읽는 법	3
시작하기	n8n이 뭔데?	4
용어	어려운 용어 → 쉬운 말 번역표	6
한눈에 보기	전체 흐름 한 줄	8
1단계	n8n 열기 (Cloud · 내 컴퓨터)	10
2단계	첫 워크플로우 (트리거 · 노드 · 실행)	14
3단계	데이터의 모양 (Item · JSON)	17
4단계	빈칸 채우기 공식 (Expression)	20
5단계	데이터 손질하기 (Edit Fields · Filter · Sort · Code)	23
6단계	갈림길 (If · Switch · Merge)	26
7단계	시작 버튼 종류 (트리거 · Publish)	28
8단계	외부 서비스 연결 (Credentials)	31
9단계	아무 서비스에나 말 걸기 (HTTP Request · API)	34
10단계	AI 직원 붙이기 (AI Agent)	37
11단계	여러 개 처리하기 (Loop · Batch · Wait)	41
12단계	사고 대비 (Executions · Pin data · Error Workflow)	43
13단계	진짜로 켜기와 관리 (Publish · 버전 · 내보내기)	45
14단계	AI 로 워크플로우 만들기 (n8n Assistant · 클로드)	48
부록 A	노드 치트시트	50
부록 B	에러 메시지 번역표	52
부록 C	자동화 설계 10계명	54

들어가며

이 자료 읽는 법

코드 없이 **블록을 이어 붙여 일을 자동으로 시키는** 도구, n8n 입문서입니다. (n8n 2.x 기준) 각 단계는 같은 순서로 되어 있습니다.

1. 📖 **이게 뭐야?** — 단계에 나오는 용어를 하나씩 풀어서 설명
2. ❓ **왜 필요해?** — 안 하면 무슨 일이 생기는지
3. 🛠️ **직접 해보기** — 실제로 클릭하는 순서와 입력값
4. ⚠️ **자주 하는 실수**
5. 💬 **AI 에게 이렇게 말하기** — n8n Assistant · 클로드에게 쓰는 복붙용 프롬프트

n8n이 뭔데?

📖 이게 뭐야?

n8n(엔에잇엔) 은 **자동화 공장의 컨베이어 벨트**입니다. "매일 아침 8시에 → 날씨를 가져와서 → AI 가 한 줄로 요약하고 → 내 텔레그램으로 보내" 같은 일을, **블록(노드)을 선으로 이어서** 만듭니다. 한 번 만들어두면 사람이 없어도 알아서 돌아갑니다.

```
[매일 08:00] ⟶ [날씨 가져오기] ⟶ [AI 요약] ⟶ [텔레그램 전송]
  트리거          노드             노드          노드
```

이름은 **n + 8글자 + n** = **n**odematio**n** (노드 + 자동화) 의 줄임말입니다.

	직접 코딩 (바이브 코딩)	n8n
만드는 방법	클로드에게 코드를 쓰게 함	화면에서 블록을 연결
결과물	앱, 웹사이트	**일을 대신 해주는 흐름**
서버·배포	내가 챙겨야 함	n8n 이 알아서 돌림
외부 서비스 연결	API 문서 읽고 코드 작성	**이미 만들어진 노드 수백 개** (Gmail, 시트, 텔레그램…)
잘하는 것	화면이 있는 제품	**반복 업무, 서비스끼리 연결, AI 비서**

비슷한 서비스로 Zapier, Make 가 있습니다. n8n 은 **내 컴퓨터·서버에 무료로 설치**할 수도 있고, 흐름 중간에 **코드와 AI 를 자유롭게** 넣을 수 있다는 점이 다릅니다.

❓ 왜 필요해?

- 매일 반복하는 일 (복사·붙여넣기, 알림 보내기, 정리하기) → **한 번 만들면 끝**
- 바이브 코딩으로 만든 앱에 **"신청 들어오면 메일 보내기"** 같은 뒤편 일을 붙이기 쉬움
- AI 를 **채팅창 밖으로** 꺼내서 실제로 일을 하게 만들 수 있음 (메일 쓰기, 시트 정리, 알림)

> **n8n = 내가 잠든 사이에도 일하는 비서를, 코드 없이 만드는 도구**

용어

어려운 용어 → 쉬운 말 번역표

원래 용어	쉬운 말	자세한 설명
Workflow (워크플로우)	**자동화 레시피 한 장**	2단계
Node (노드)	**블록 하나** (한 가지 일)	2단계
Trigger (트리거)	**시작 버튼** (언제 시작할지)	2, 7단계
Canvas (캔버스)	**블록을 놓는 작업판**	2단계
Execute (실행)	**한 번 돌려보기**	2단계
Item (아이템)	**상자 하나** (데이터 한 묶음)	3단계
JSON	**이름표 붙은 데이터**	3단계
Expression `{{ }}`	**빈칸 채우기 공식**	4단계
Edit Fields (Set)	**데이터 손질 블록**	5단계
If / Switch	**갈림길**	6단계
Merge	**합류 지점**	6단계

원래 용어	쉬운 말	자세한 설명
Webhook (웹훅)	**초인종** (누가 누르면 시작)	7단계
Publish (게시)	**"이제 진짜로 켜기"**	7, 13단계
Credential (자격 증명)	**열쇠 보관함**	8단계
API	**서비스끼리 대화하는 창구**	9단계
HTTP Request	**아무 창구에나 말 걸기**	9단계
AI Agent	**도구를 쓸 줄 아는 AI 직원**	10단계
Chat Model	**AI 의 두뇌**	10단계
Memory	**AI 의 기억**	10단계
Tool (도구)	**AI 가 쓸 수 있는 연장**	10단계
Loop / Batch	**나눠서 여러 번**	11단계
Execution (실행 기록)	**블랙박스 영상**	12단계
Pin data	**테스트용 데이터 고정**	12단계
Error Workflow	**사고 신고 센터**	12단계

한눈에 보기

전체 흐름 한 줄

```
n8n 열기 → 첫 워크플로우 → 데이터 모양(item) → 빈칸 공식({{ }}) → 데이터 손질 → 갈림길
→ 시작 버튼 종류(트리거) → 서비스 연결(열쇠) → 아무 API 부르기 → AI 직원 붙이기
→ 여러 개 처리 → 사고 대비 → 진짜로 켜기(Publish) → AI 로 워크플로우 만들기
```

구간	단계	내용
준비	1단계	n8n 열기 (Cloud / 내 컴퓨터)
기본기	2~6단계	노드, 데이터, 표현식, 갈림길
바깥과 연결	7~9단계	트리거, 외부 서비스, API
AI	10단계	AI Agent
실전 운영	11~13단계	반복, 에러, Publish
더 빠르게	14단계	AI 로 워크플로우 만들기

모든 자동화는 이 세 칸으로 설계합니다

① 언제? (트리거)	→	② 무엇을? (처리)	→	③ 어디로? (행동)
매일 아침 8시		날씨 가져와서 요약		텔레그램으로 보내기
폼이 제출되면		신청 내용 정리		시트에 저장 + 메일 발송
채팅이 오면		AI 가 답변 생성		채팅으로 답장

블록을 놓기 전에 **종이에 이 세 칸부터** 적습니다. 이게 안 되면 n8n 을 켜도 헤맵니다.

1단계

n8n 열기 (Cloud · 내 컴퓨터)

📖 이게 뭐야?

n8n 을 쓰는 방법은 두 가지입니다.

	n8n Cloud (추천)	내 컴퓨터에 설치 (Self-hosted)
비용	**14일 무료 체험** (카드 불필요) → 이후 유료	**무료** (Community Edition)
설치	없음. 가입만	터미널에서 `npx n8n`
인터넷 주소	자동으로 생김 (`https://내이름.app.n8n.cloud`)	`http://localhost:5678` (나만 접속)
Google 연결	**버튼 한 번**	Google Cloud 에서 직접 설정 (20분+)
텔레그램 트리거	바로 됨	공개 HTTPS 주소가 필요해서 **안 됨**
컴퓨터를 끄면	계속 돌아감	**멈춤**

> 수업은 **Cloud 무료 체험**으로 합니다. 체험이 끝나면 워크스페이스가 삭제되니, 남기고 싶은 워크플로우는 **다운로드**(13단계)해두고 내 컴퓨터 n8n 에 가져오면 됩니다.

Cloud 체험 한도: 1,000번 실행, Pro 플랜 기능, 14일. 체험 종료 후 **90일 안에** 워크플로우를 다운로드할 수 있습니다.

🛠️ 직접 해보기

방법 A — Cloud (수업용)

1. **n8n.io** → **Get started for free** → 이메일로 가입
2. 워크스페이스 이름 입력 (주소가 됨: `이름.app.n8n.cloud`)
3. 몇 가지 설문에 답하면 → 첫 화면 (Overview)

방법 B — 내 컴퓨터 (수업 후 계속 쓰고 싶을 때) Node.js **20.19 ~ 24** 버전이 필요합니다.

```
node --version      # v20.19 이상, v24 이하인지 확인
npx n8n             # 설치 없이 바로 실행 (처음엔 다운로드로 몇 분 걸림)
```

→ 브라우저에서 **http://localhost:5678** → 관리자 계정 만들기. 끄려면 터미널에서 **Ctrl + C**.

Docker 를 쓴다면:

```
docker volume create n8n_data
docker run -it --rm --name n8n -p 5678:5678 -e GENERIC_TIMEZONE="Asia/Seoul" -e TZ="Asia/Seoul" -v n8n_data:/home/node/.n8n n8nio/n8n
```

📖 화면 구성

위치	이름	하는 일
왼쪽 메뉴	**Overview**	내 워크플로우·열쇠·실행 기록 한곳에
Overview 탭	**Workflows**	워크플로우 목록
Overview 탭	**Credentials**	연결해둔 서비스 열쇠 목록 (8단계)
Overview 탭	**Executions**	모든 실행 기록 (12단계)
Overview 탭	**Data tables**	n8n 안에 만드는 간단한 표 (엑셀처럼)
오른쪽 위	**Create workflow**	새 레시피 만들기
어디서나	**n8n Assistant** (Ctrl/Cmd + J)	n8n 안의 AI 도우미 (14단계)

⚠️ 자주 하는 실수

- 회사·학교 Google 계정으로 가입 → 나중에 Gmail 연결이 관리자 정책에 막힘. **개인 Gmail** 추천
- `npx n8n` 을 켜둔 터미널을 닫음 → n8n 도 같이 꺼짐
- Node.js 가 너무 옛날/최신 버전 → `node --version` 먼저

💬 AI 에게 이렇게 말하기

```
n8n 을 처음 써봐. 지금 보이는 화면에서 Workflows, Credentials, Executions 가 각각 뭘 하는 곳인지
초보자에게 설명하듯 한 줄씩 알려줘.
```

2단계

첫 워크플로우 (트리거 · 노드 · 실행)

📖 이게 뭐야?

Workflow (워크플로우) = 자동화 레시피 한 장 블록들을 선으로 연결해놓은 한 장의 그림입니다. 하나의 워크플로우 = 하나의 자동화.

Node (노드) = 블록 하나 한 가지 일만 하는 블록입니다. "메일 보내기", "시트에 한 줄 추가", "AI 에게 묻기" 같은.

Trigger (트리거) = 시작 버튼 워크플로우의 **맨 앞에 오는 특별한 노드**. "언제 시작할지"를 정합니다. 모든 워크플로우는 트리거로 시작합니다.

트리거	언제 시작?
Trigger manually	내가 실행 버튼을 누를 때 (연습용)
On a schedule	정해진 시간마다
On form submission	n8n 이 만들어준 웹 폼이 제출되면
On chat message	채팅 메시지가 오면
On webhook call	누가 특정 주소를 호출하면
On app event	Gmail 새 메일, 텔레그램 메시지 등

노드 설정창 (더블클릭하면 열림)



가운데에서 설정 → **Execute step** 을 누르면 → 오른쪽 OUTPUT 에 결과가 보입니다.

자동 저장 n8n 2.x 는 수정하면 **1~5초 안에 자동 저장**됩니다. 따로 저장 버튼이 없습니다.

🛠️ 직접 해보기

1. **Create workflow** → 이름을 `01-첫 워크플로우` 로 (왼쪽 위 제목 클릭)
2. 가운데 **+ (Add first step)** → **Trigger manually**
3. 트리거 오른쪽의 **+** → 검색창에 `Edit Fields` → **Edit Fields (Set)**
4. **Add Field** → 이름 `message`, 타입 String, 값 `안녕 n8n!`
5. 설정창 위의 **Execute step** → 오른쪽 OUTPUT 에 `message: 안녕 n8n!` 확인
6. 설정창 닫기 → 캔버스 아래의 **Execute workflow** → 모든 노드에 **초록 체크** ✅

❓ 왜 필요해?

n8n 의 모든 자동화는 이 구조의 반복입니다. **트리거 → 노드 → 노드 → ...** 블록을 하나 추가할 때마다 **Execute step 으로 바로 확인**하는 습관이 가장 중요합니다.

⚠️ 자주 하는 실수

- 노드를 10개 다 만든 다음에 처음 실행 → 어디서 틀렸는지 모름. **하나 놓고 → 실행 → 확인**
- 선이 안 이어진 노드 → 실행되지 않음 (회색으로 따로 떠 있음)
- 노드 이름을 안 바꿈 → `Edit Fields1`, `Edit Fields2`... 나중에 헷갈림. **노드 이름 = 하는 일** (`인사말 만들기`)

💬 AI 에게 이렇게 말하기

```
n8n 에서 Manual Trigger 다음에 Edit Fields 노드로 name 과 age 필드를 만드는 방법을
클릭 순서대로 알려줘.
```

3단계

데이터의 모양 (Item · JSON)

📖 이게 뭐야?

JSON = 이름표 붙은 데이터

```
{ "name": "홍길동", "age": 20, "city": "서울" }
```

`이름표: 값` 의 묶음입니다. n8n 의 모든 데이터는 이 모양으로 흐릅니다.

Item (아이템) = 상자 하나 노드와 노드 사이에는 **상자들이 줄지어** 지나갑니다. 상자 하나 = item 하나 = JSON 한 묶음.

```
[
  { "name": "홍길동", "age": 20 },     ← item 1
  { "name": "김철수", "age": 25 },     ← item 2
  { "name": "이영희", "age": 23 }      ← item 3
]
```

🔑 **n8n 의 제일 중요한 규칙: 노드는 item 마다 한 번씩 일한다** 상자가 3개 들어오면 "메일 보내기" 노드는 **메일을 3통** 보냅니다. 반복문을 따로 만들 필요가 없습니다.

OUTPUT 을 보는 3가지 방법 (설정창 오른쪽 위)

보기	생김새	언제
Schema	필드 이름 목록	어떤 이름표가 있는지 볼 때 (드래그해서 쓰기 좋음)
Table	엑셀 같은 표	item 이 여러 개일 때
JSON	원래 모양	안쪽에 또 묶음이 있는 복잡한 데이터

안쪽 묶음 (중첩)

```
{ "user": { "name": "홍길동", "address": { "city": "서울" } }, "tags": ["학생", "개발"] }
```

- `{ }` = 이름표 묶음 (객체), `[ ]` = 순서 있는 목록 (배열, 0번부터 셈)
- `user` 안의 `address` 안의 `city` → `user.address.city`
- `tags` 의 첫 번째 → `tags[0]` = `"학생"`

🛠️ 직접 해보기

1. Edit Fields 노드의 **Mode** 를 **JSON** 으로 바꾸고 아래를 붙여넣기

```
{ "students": [ { "name": "홍길동", "score": 85 }, { "name": "김철수", "score": 62 }, { "name": "이영희", "score": 93 } ] }
```

2. Execute step → OUTPUT 을 Schema / Table / JSON 으로 바꿔가며 보기 → item **1개** (students 목록이 통째로 1개)

3. 다음 노드로 **Split Out** 추가 → Fields To Split Out: `students` → Execute step → item **3개**로 쪼개짐 ✅

❓ 왜 필요해?

"왜 메일이 1통만 갔지?", "왜 10번 갔지?" — n8n 에서 생기는 문제의 대부분은 **item 개수**를 몰라서 생깁니다. OUTPUT 위에 적힌 **"3 items"** 를 항상 확인하세요.

⚠️ 자주 하는 실수

- 목록이 item 하나 안에 통째로 들어 있는데 여러 개라고 착각 → **Split Out** 으로 쪼개기
- 반대로 여러 item 을 하나로 합쳐야 할 때 (한 메일에 전체 목록) → **Aggregate** 노드
- 배열 번호를 1부터 셈 → n8n(자바스크립트)은 **0부터**

💬 AI 에게 이렇게 말하기

```
n8n 노드 OUTPUT 이 아래 JSON 이야. 이게 item 몇 개인지, 그리고 학생 이름만 꺼내려면
어떤 경로(예: students[0].name)를 써야 하는지 알려줘.
(여기에 JSON 붙여넣기)
```

4단계

빈칸 채우기 공식 (Expression)

📖 이게 뭐야?

Expression (표현식) = 빈칸 채우기 공식 노드 설정 칸에 **고정된 글자 대신 앞에서 온 데이터**를 넣는 방법입니다. `{{ }}` 안에 적습니다.

```
고정값:   안녕하세요 홍길동님
표현식:   안녕하세요 {{ $json.name }}님     ← item 마다 이름이 바뀜
```

Fixed / Expression 스위치 설정 칸에 마우스를 올리면 **Fixed | Expression** 선택이 있습니다. Expression 으로 바꿔야 `{{ }}` 가 작동합니다. 가장 쉬운 방법: 왼쪽 INPUT 의 필드를 **끌어다 칸에 놓기** (드래그 앤 드롭) → 자동으로 표현식이 됩니다.

자주 쓰는 공식

공식	뜻	결과 예
`{{ $json.name }}`	바로 앞 노드의 `name`	홍길동
`{{ $json.user.address.city }}`	안쪽 묶음	서울
`{{ $json.tags[0] }}`	목록의 첫 번째	학생
`{{ $('신청서').item.json.email }}`	**이름이 '신청서'인 노드**의 값 (바로 앞이 아니어도)	a@b.com
`{{ $now.toFormat('yyyy-MM-dd') }}`	오늘 날짜	2026-10-14
`{{ $now.toFormat('HH:mm') }}`	지금 시각	08:00
`{{ $json.name.toUpperCase() }}`	대문자로	HONG
`{{ $json.score >= 80 ? '합격' : '불합격' }}`	조건에 따라 다른 값	합격
`{{ $json.price * 1.1 }}`	계산	11000

미리보기 표현식 칸 아래에 **결과가 바로 보입니다**. 빨간 글씨면 틀린 것.

🛠️ 직접 해보기

3단계의 Split Out 다음에 **Edit Fields** 추가 → Add Field:

- `greeting` = `{{ $json.name }}님, 점수는 {{ $json.score }}점입니다`
- `result` = `{{ $json.score >= 80 ? '합격' : '재시험' }}`
- `checked_at` = `{{ $now.toFormat('yyyy-MM-dd HH:mm') }}`

→ Execute step → Table 보기에서 3명 각자 다른 문장이 나오면 성공

❓ 왜 필요해?

표현식이 없으면 모든 메일이 똑같은 내용으로 갑니다. **"홍길동님 안녕하세요"** 처럼 사람마다 다른 내용을 만드는 게 자동화의 핵심입니다.

⚠️ 자주 하는 실수

- Fixed 모드에 `{{ }}` 를 적음 → 글자 그대로 `{{ $json.name }}` 이 메일에 찍힘
- 노드 이름을 바꾸면 `$('예전이름')` 공식이 깨짐 → 노드 이름은 **일찍 정하고** 바꾸지 않기
- 필드 이름 대소문자·띄어쓰기 틀림 (`Name` ≠ `name`). 띄어쓰기가 있으면 `{{ $json['First Name'] }}`

💬 AI 에게 이렇게 말하기

```
n8n 표현식을 만들어줘. 입력 item 은 {"name":"홍길동","score":85,"joined":"2026-03-02"} 이야.
"홍길동님 (가입 225일째) - 합격" 처럼 가입 후 지난 날짜 수와 80점 이상이면 합격을 넣고 싶어.
```

5단계

데이터 손질하기 (Edit Fields · Filter · Sort · Code)

📖 이게 뭐야?

노드	쉬운 말	하는 일
Edit Fields (Set)	손질 블록	필드 만들기·이름 바꾸기·필요한 것만 남기기
Filter	체	조건에 맞는 item 만 통과
Sort	줄 세우기	점수순, 날짜순 정렬
Limit	앞에서 N개만	상위 3개만
Remove Duplicates	중복 제거	같은 이메일 두 번 방지
Split Out	쪼개기	목록 1개 → item 여러 개
Aggregate	모으기	item 여러 개 → 목록 1개
Code	만능 칼	자바스크립트로 직접 처리 (최후의 수단)

Edit Fields 의 "Include Other Input Fields" 꺼두면 → **내가 만든 필드만** 다음으로 넘어감 (깔끔). 켜두면 → 원래 필드 + 새 필드.

🛠️ 직접 해보기

4단계에 이어서:

1. **Filter** 추가 → Conditions: `{{ $json.score }}` **is greater than or equal to** `80` → 2명만 통과
2. **Sort** 추가 → Field Name: `score`, Order: Descending → 높은 점수부터
3. **Aggregate** 추가 → Individual Fields: `name` → item 1개 안에 `name: ["이영희", "홍길동"]`
4. 마지막 **Edit Fields** → `summary` = `{{ $json.name.join(', ') }} 합격!`

Code 노드는 언제? 노드로 3개 이상 이어야 하는 복잡한 변환일 때만. 클로드에게 맡기면 됩니다.

```
// Code 노드 (Run Once for All Items) — 점수 평균 구하기
const scores = $input.all().map(item => item.json.score);
const avg = scores.reduce((a, b) => a + b, 0) / scores.length;
return [{ json: { average: Math.round(avg) } }];
```

❓ 왜 필요해?

외부 서비스에서 온 데이터는 **필드가 수십 개**입니다. 필요한 것만 남기고 이름을 알아보기 쉽게 바꿔야 뒤의 노드(특히 AI)가 헷갈리지 않습니다.

⚠️ 자주 하는 실수

- 처음부터 Code 노드로 다 해결 → 다른 사람이(미래의 내가) 못 읽음. **기본 노드 먼저**
- Code 노드에서 `return` 모양 틀림 → 반드시 `[{ json: {...} }]` 처럼 **목록 안에 json**
- Filter 의 숫자 비교를 글자(String) 조건으로 함 → "100" < "80" 처럼 이상하게 비교됨. **Number 조건** 선택

💬 AI 에게 이렇게 말하기

```
n8n Code 노드(JavaScript, Run Once for All Items)를 써줘.
입력 item 들은 {"name":..., "score":...} 모양이야. 평균보다 높은 사람만 남기고 각 item 에 rank(순위) 필드를 붙여서 돌려줘.
반환 형식은 n8n 규칙대로 [{ json: {...} }].
```

6단계

갈림길 (If · Switch · Merge)

📖 이게 뭐야?

If = 두 갈래 길 조건이 맞으면 **true** 출구, 아니면 **false** 출구로 item 이 갑니다.

```
                ┌─ true  → [축하 메일]
[점수 확인] ──If─┤
                └─ false → [재시험 안내 메일]
```

Switch = 여러 갈래 길 값에 따라 3개 이상으로 나눕니다. (예: 문의 종류가 `결제` / `배송` / `기타`)

Merge = 합류 지점 갈라진 길이나 **두 곳에서 온 데이터를 하나로** 합칩니다.

Merge 방식	언제
Append	두 줄을 그냥 이어 붙이기
Combine → Matching Fields	같은 이메일끼리 짝지어 합치기 (엑셀 VLOOKUP 같은)
Combine → Position	1번은 1번끼리, 2번은 2번끼리
Choose Branch	둘 다 끝날 때까지 기다렸다가 한쪽 데이터만

🛠️ 직접 해보기

Split Out 다음에 (Filter 대신) **If** 추가:

1. Conditions: `{{ $json.score }}` **Number → is greater than or equal to** `80`
2. Execute step → OUTPUT 에서 **True Branch (2 items)** / **False Branch (1 item)** 탭 확인
3. true 출구 → Edit Fields `msg` = `🎉 {{ $json.name }} 합격`
4. false 출구 → Edit Fields `msg` = `📚 {{ $json.name }} 재시험`
5. 두 Edit Fields 를 **Merge** (Append) 로 합치기 → 3개 다시 모임

❓ 왜 필요해?

현실의 일은 늘 **"경우에 따라"** 다릅니다. VIP 고객이면 담당자에게 알림, 아니면 자동 답장. 갈림길이 없으면 모두에게 같은 행동을 하게 됩니다.

⚠️ 자주 하는 실수

- If 의 **false 출구에 아무것도 안 연결** → 그쪽 item 은 조용히 사라짐 (의도한 거면 OK)
- 숫자를 **String 조건**으로 비교 → 조건 종류를 Number 로
- 빈 값 체크는 **is empty / is not empty** 조건 사용 (`== ""` 로 하면 null 을 못 잡음)

💬 AI 에게 이렇게 말하기

```
n8n 에서 문의 내용(text 필드)에 "환불"이 들어 있으면 담당자에게 텔레그램, "배송"이면 배송팀 시트에 기록,
나머지는 자동 답장 메일을 보내고 싶어. Switch 노드 규칙을 어떻게 설정하는지 알려줘.
```

7단계

시작 버튼 종류 (트리거 · Publish)

📖 이게 뭐야?

Schedule Trigger = 알람 시계

- Trigger Interval: Minutes / Hours / **Days** / Weeks / Months / Custom (Cron)
- 예: Days, Trigger at Hour `8am`, Minute `0` → 매일 08:00
- ⚠️ 시간대는 워크플로우 **Settings → Timezone** 을 `Asia/Seoul` 로 확인

Form Trigger = n8n 이 만들어주는 신청서 코드 없이 웹 폼이 생깁니다. Form Title, Form Description, **Form Elements** (Text, Email, Number, Dropdown, Date, Textarea, File...)

Chat Trigger = 채팅창 n8n 이 채팅창을 만들어줍니다. AI 챗봇(10단계)과 짝꿍.

Webhook = 초인종 고유한 주소가 생기고, 누가 그 주소로 요청을 보내면 시작합니다. 다른 앱(바이브 코딩으로 만든 내 앱!)과 연결할 때 씁니다.

App 트리거 — Gmail(새 메일), Telegram(새 메시지), Google Sheets(새 행) 등

🔑 **Test URL vs Production URL** Form · Webhook · Chat 트리거는 주소가 **두 개**입니다.

	Test URL	Production URL
언제 작동?	**Execute step / Listen for test event** 를 누른 동안 **한 번**	**Publish 한 뒤 항상**
결과 보기	편집 화면에 바로 보임	**Executions** 탭에서 확인
용도	만드는 중 테스트	실제 사용자에게 공유

Publish = "이제 진짜로 켜기" n8n 2.x 는 예전의 Active 스위치 대신 **Publish 버튼**(오른쪽 위)을 씁니다.

- 수정은 계속 **초안(draft)** 으로 자동 저장되고, **Publish 를 눌러야** 실제로 돌아가는 버전이 바뀝니다
- Publish 하기 전까지 Schedule·App 트리거는 **혼자 돌지 않습니다**
- 단축키 **Shift + P**. 끄기(Unpublish)는 Publish 옆 ▾ 메뉴

🛠️ 직접 해보기 — 3분 신청서

1. 새 워크플로우 → **On form submission**
2. Form Title `수업 신청`, Form Elements: `이름` (Text, Required), `이메일` (Email, Required), `관심 분야` (Dropdown: 웹, 앱, AI)
3. **Execute step** → 아래 **Test URL** 이 열린 폼에서 제출 → OUTPUT 에 내가 쓴 값 ✅
4. 다음 노드로 Edit Fields `welcome` = `{{ $json['이름'] }}님 신청 완료! 관심 분야: {{ $json['관심 분야'] }}`
5. 오른쪽 위 **Publish** → Form Trigger 를 열어 **Production URL** 로 바꿔 복사 → 친구에게 보내기
6. 친구가 제출 → Overview → **Executions** 에서 확인

❓ 왜 필요해?

트리거를 바꾸면 **같은 처리 흐름을 완전히 다른 상황에** 쓸 수 있습니다. 연습은 Manual 로, 실제 운영은 Schedule·Form·App 트리거 + Publish 로.

⚠️ 자주 하는 실수

- **Test URL 을 친구에게 공유** → 내가 Execute 를 누르고 있을 때만 작동. 공유는 **Production URL**
- Publish 안 하고 "왜 아침 8시에 안 왔지?" → 오른쪽 위가 **Published** 인지 확인
- 수정한 뒤 Publish 를 다시 안 누름 → 실제로는 **예전 버전**이 계속 돌고 있음 (버튼에 "has changes" 표시)
- 한글 필드 이름은 `{{ $json['이름'] }}` 처럼 **대괄호+따옴표**로

💬 AI 에게 이렇게 말하기

```
n8n Schedule Trigger 로 평일(월~금) 오전 8시 30분에만 실행되게 하고 싶어.
Custom(Cron) 표현식과 Timezone 설정을 알려줘.
```

8단계

외부 서비스 연결 (Credentials)

📖 이게 뭐야?

Credential (자격 증명) = 열쇠 보관함 n8n 이 내 Gmail·시트·텔레그램을 대신 쓰려면 **열쇠**가 필요합니다. 열쇠는 Credentials 에 **한 번만** 등록하고, 여러 워크플로우에서 같이 씁니다. 열쇠 내용은 암호화되어 저장되고, 워크플로우를 내보내도(13단계) **열쇠는 같이 나가지 않습니다.**

열쇠의 종류

방식	쉬운 말	예	만드는 법
OAuth2	"이 앱에 권한 줄게요" 버튼	Gmail, Google Sheets	**Sign in with Google** 클릭 (Cloud)
API Key	긴 비밀번호 문자열	Gemini, OpenAI	서비스 사이트에서 발급 → 붙여넣기
Bot Token	봇 전용 비밀번호	Telegram	BotFather 에게 받기

🛠️ 직접 해보기

Google (Gmail · Sheets) — Cloud 기준

1. 노드(예: Gmail) 추가 → Credential to connect with: **Create new credential**
2. **Sign in with Google** → 계정 선택 → 권한 허용 → 저장
3. 같은 방식으로 Google Sheets 열쇠도 (서비스마다 따로)

Telegram 봇

1. 텔레그램 앱에서 **@BotFather** 검색 → `/newbot` → 봇 이름 → 봇 아이디(끝이 `bot`)
2. 받은 **토큰**(`1234567:ABC...`) 복사
3. n8n Telegram 노드 → Create new credential → Access Token 에 붙여넣기 → Save
4. 내 봇에게 먼저 아무 메시지나 보내기 (봇은 먼저 말 건 사람에게만 보낼 수 있음)

Chat ID 알아내기 (텔레그램으로 "나에게" 보내려면 필요) Telegram Trigger(On message) 를 놓고 Execute step → 봇에게 메시지 보내기 → OUTPUT 의 `message.chat.id` 숫자

Gemini API 키 (AI 용, 무료 사용량 있음)

1. **aistudio.google.com/apikey** → **Create API Key** → 복사
2. n8n 에서 Google Gemini Chat Model 노드 → Create new credential (Google Gemini(PaLM) API) → API Key 붙여넣기 (Host 는 기본값 `https://generativelanguage.googleapis.com` 그대로)

❓ 왜 필요해?

열쇠를 노드마다 따로 넣으면 바꿀 때 전부 고쳐야 하고, 비밀번호가 여기저기 흩어집니다. Credentials 한곳에서 관리하면 **열쇠 하나 바꾸면 전체가 바뀝니다.**

⚠️ 자주 하는 실수

- API 키·봇 토큰을 **Edit Fields 나 표현식에 직접** 적음 → 워크플로우를 공유할 때 키가 같이 나감. **반드시 Credential 에**
- Google 연결 시 권한 체크박스를 **해제** → 나중에 "insufficient permission" 에러
- 텔레그램 봇에게 먼저 말을 안 걸고 Send Message → `chat not found`
- 키가 유출됐다면 → **그 서비스에서 즉시 폐기·재발급** 후 Credential 업데이트

📦 더 알고 싶은 사람만 (몰라도 실습 가능)

내 컴퓨터(Self-hosted) n8n 에서 Google 연결하기: Cloud 의 "Sign in with Google" 간편 연결은 내 컴퓨터 n8n 에서는 쓸 수 없습니다. Google Cloud Console 에서 ① 프로젝트 만들기 ② Gmail API / Sheets API / Drive API 켜기 ③ OAuth 동의 화면 설정 ④ OAuth 클라이언트(웹 애플리케이션) 만들기 + 리디렉션 URI 에 n8n 이 보여주는 주소 (`http://localhost:5678/rest/oauth2-credential/callback`) 등록 ⑤ Client ID / Secret 을 n8n 에 입력 — 순서로 진행합니다.

💬 AI 에게 이렇게 말하기

```
n8n 에서 Telegram Send Message 노드를 쓰려는데 "Bad Request: chat not found" 에러가 나.
원인과 Chat ID 를 확인하는 방법을 순서대로 알려줘.
```

9단계

아무 서비스에나 말 걸기 (HTTP Request · API)

📖 이게 뭐야?

API = 서비스끼리 대화하는 창구 날씨 서비스, 환율 서비스, 내 앱… 대부분의 서비스는 **"이 주소로 물어보면 JSON 으로 답해줄게"** 라는 창구(API)를 열어둡니다.

HTTP Request 노드 = 아무 창구에나 말 걸기 n8n 에 전용 노드가 없는 서비스도 이 노드 하나로 연결할 수 있습니다.

설정	뜻	예
Method	무슨 요청?	`GET` 가져오기 / `POST` 보내기
URL	창구 주소	`https://api.open-meteo.com/v1/forecast`
Query Parameters	주소 뒤 `?a=1&b=2` 질문 내용	`latitude=37.57`
Headers	봉투에 적는 정보 (인증 등)	`Authorization: Bearer ...`
Body	POST 로 보낼 내용	JSON

cURL 붙여넣기 API 문서에 `curl ...` 예시가 있으면 → HTTP Request 노드의 **Import cURL** 에 붙여넣기 → 칸이 자동으로 채워집니다.

🛠️ 직접 해보기 — 서울 날씨 (키 필요 없음)

1. HTTP Request 노드 → Method `GET`
2. URL:

```
https://api.open-meteo.com/v1/forecast?latitude=37.5665&longitude=126.978&current=temperature_2m&daily=temperature_2m_max,temperature_2m_min,precipitation_probability_max&timezone=Asia%2FSeoul&forecast_days=1
```

3. Execute step → OUTPUT (JSON 보기)
4. Edit Fields 로 꺼내기:
   - `지금` = `{{ $json.current.temperature_2m }}°C`
   - `최고` = `{{ $json.daily.temperature_2m_max[0] }}°C`
   - `최저` = `{{ $json.daily.temperature_2m_min[0] }}°C`
   - `비올확률` = `{{ $json.daily.precipitation_probability_max[0] }}%`

❓ 왜 필요해?

전용 노드는 수백 개지만 세상의 API 는 수만 개입니다. HTTP Request 를 쓸 줄 알면 **연결 못 하는 서비스가 거의 없습니다.** 바이브 코딩으로 만든 내 앱의 API 도 이걸로 부릅니다.

⚠️ 자주 하는 실수

- 인증이 필요한 API 키를 URL 에 그대로 붙임 → **Authentication → Generic Credential Type** (Header Auth 등)으로 열쇠 보관함에
- 응답이 목록인데 item 1개로 들어옴 → **Split Out** 으로 쪼개기 (3단계)
- 너무 자주 호출 → `429 Too Many Requests` → 11단계 Wait / Batch

💬 AI 에게 이렇게 말하기

```
이 API 문서의 curl 예시를 n8n HTTP Request 노드 설정(Method, URL, Query, Header, Body)으로 바꿔줘.
API 키는 Header Auth credential 로 넣는 방법도 알려줘.
(여기에 curl 붙여넣기)
```

10단계

AI 직원 붙이기 (AI Agent)

📖 이게 뭐야?

AI Agent 노드 = 도구를 쓸 줄 아는 AI 직원 그냥 AI 에게 묻는 것과 다르게, **스스로 판단해서 도구(시트 읽기, 메일 보내기, 검색...)를 골라 씁니다.**

AI Agent 노드 아래에는 **부품 꽂는 자리**가 있습니다.



부품	쉬운 말	없으면?
Chat Model	두뇌	작동 안 함 (필수)
Memory	대화 기억	매 메시지마다 처음 보는 사람처럼 대답
Tool	연장	말만 하고 실제 일은 못 함 (n8n 문서상 1개 이상 연결)

System Message = 직원 업무 지침서 AI Agent → Options → **System Message** 에 "너는 누구고, 뭘 하고, 뭘 하면 안 되는지" 를 적습니다. 바이브 코딩의 CLAUDE.md 와 같은 역할입니다.

Prompt (User Message)

- Chat Trigger 와 연결하면 → **Connected Chat Trigger Node** (채팅 내용이 그대로 질문)
- 다른 트리거면 → **Define below** 에 `{{ }}` 로 질문을 만들어 넣기

도구에게 AI 가 값을 채우게 하기 ✨ Tool 노드의 칸 옆 ✨**(Let the model define this parameter)** 를 누르면, 그 칸은 AI 가 상황에 맞게 채웁니다. (내부적으로 `{{ $fromAI(...) }}` 로 바뀜)

AI Agent 노드 vs n8n Agents 최근 n8n 에는 워크플로우 밖에서 따로 만드는 **Agents** 탭(미리보기 기능)도 생겼습니다. 이 강의는 어디서나 쓰이는 **AI Agent 노드**로 배웁니다. 원리는 같습니다.

🛠️ 직접 해보기 — 우리 반 FAQ 챗봇

1. 새 워크플로우 → **On chat message** (Chat Trigger)
2. **+** → **AI Agent**
3. AI Agent 아래 **Chat Model +** → **Google Gemini Chat Model** → 8단계에서 만든 Gemini 열쇠 → 모델 목록에서 **flash** 계열 선택
4. **Memory +** → **Simple Memory** (Context Window Length `5`)
5. **Tool +** → **Google Sheets Tool** → 미리 만든 `FAQ` 시트(질문 | 답변 열) 연결, Operation: Get Row(s)
6. AI Agent → Options → **System Message**:

```
너는 '우리 반 도우미'야.
- 수업 관련 질문은 반드시 FAQ 시트 도구를 먼저 찾아보고 거기 있는 답변으로 대답해.
- 시트에 없는 내용은 지어내지 말고 "강사님께 확인해볼게요" 라고 말해.
- 존댓말, 3문장 이내.
```

7. 캔버스 아래 **Open chat** → "점심시간이 언제예요?" → AI Agent 를 열어 **어떤 도구를 썼는지** 로그 확인

❓ 왜 필요해?

AI 를 **채팅창 밖으로** 꺼내는 단계입니다. AI 가 시트를 읽고, 메일을 쓰고, 알림을 보내면 그때부터 **진짜 일을 하는 비서**가 됩니다.

⚠️ 자주 하는 실수

- System Message 없이 → AI 가 모르는 것도 **그럴듯하게 지어냄**. "모르면 모른다고 해" 를 꼭 적기
- 도구 이름·설명을 대충 → AI 가 언제 그 도구를 써야 할지 모름. 도구 노드 이름을 `FAQ 시트 검색` 처럼
- Memory 없이 챗봇 → "아까 말한 거" 를 못 알아들음
- 무료 API 한도 초과 → `429` / quota 에러. 잠시 후 다시, 또는 다른 모델
- **메일 보내기·삭제 같은 도구**를 AI 에게 바로 줌 → 실수로 보낼 수 있음. 처음엔 **읽기 도구만**

📦 더 알고 싶은 사람만 (몰라도 실습 가능)

대화가 아니라 "한 번 처리" 일 때: 이메일 요약·분류처럼 도구가 필요 없는 일은 AI Agent 대신 **Basic LLM Chain** 이나 Gemini / OpenAI / Anthropic **앱 노드의 Message 동작**이 더 단순하고 빠릅니다.

정해진 모양으로 답받기: AI Agent 의 **Require Specific Output Format** 을 켜고 **Structured Output Parser** 를 연결하면 `{"category": "환불", "urgent": true}` 처럼 항상 같은 JSON 으로 받아 다음 노드(If 등)에서 바로 쓸 수 있습니다.

💬 AI 에게 이렇게 말하기

```
n8n AI Agent 의 System Message 를 써줘. 역할: 쇼핑몰 고객 문의 분류 직원.
문의를 환불/배송/상품/기타 중 하나로 분류하고, 급한지(true/false) 판단해서 JSON 으로만 답하게.
모르는 건 지어내지 않게 규칙도 넣어줘.
```

11단계

여러 개 처리하기 (Loop · Batch · Wait)

📖 이게 뭐야?

3단계에서 배운 대로 n8n 노드는 **item 마다 알아서 반복**합니다. 그래서 대부분은 반복문이 필요 없습니다. 그런데 이럴 땐 **나눠서** 해야 합니다.

- API 가 "1초에 5번까지만" 같은 **속도 제한**이 있을 때
- AI 에게 100개를 한 번에 보내면 **시간 초과**가 날 때

Loop Over Items = 나눠서 여러 번 item 을 **Batch Size** 개씩 묶어서 차례로 처리합니다. 출구가 두 개입니다.



Wait = 잠깐 쉬기 지정한 시간만큼 멈췄다가 다음으로 갑니다. 속도 제한 피하기, "24시간 뒤에 리마인드 메일" 등.

🛠️ 직접 해보기

1. Manual Trigger → Code 노드로 item 20개 만들기

```
return Array.from({ length: 20 }, (_, i) => ({ json: { no: i + 1 } }));
```

2. **Loop Over Items** (Batch Size `5`)
3. loop 출구 → Edit Fields `msg` = `{{ $json.no }}번 처리` → **Wait** (2 Seconds) → 선을 다시 Loop Over Items 입력으로
4. done 출구 → Edit Fields `msg` = `끝!`
5. Execute workflow → 5개씩 4번 도는 걸 관찰

❓ 왜 필요해?

처음엔 3개로 테스트해서 잘 되던 게, 실제로 500개가 들어오면 **속도 제한·시간 초과**로 터집니다. Cloud 체험판은 한 번 실행이 **180초**를 넘으면 멈춥니다.

⚠️ 자주 하는 실수

- 반복이 필요 없는데 Loop 부터 씀 → 그냥 노드를 이으면 item 마다 돈다
- loop 출구의 마지막 노드를 다시 **Loop Over Items 로 안 이음** → 첫 묶음만 처리하고 끝남
- done 출구에서 처리된 결과를 기대 → done 은 **전체가 끝났다는 신호**. 결과를 모으려면 Aggregate

💬 AI 에게 이렇게 말하기

```
n8n 에서 Google Sheets 의 300행을 하나씩 Gemini 로 요약하는데 429 에러가 나.
Loop Over Items 와 Wait 로 분당 15회 이하로 맞추는 구조를 노드 순서대로 설명해줘.
```

12단계

사고 대비 (Executions · Pin data · Error Workflow)

📖 이게 뭐야?

Executions = 블랙박스 영상 워크플로우가 돌 때마다 기록이 남습니다. 워크플로우 위쪽 **Executions** 탭 → 실행 하나 클릭 → **그때 각 노드에 무슨 데이터가 있었는지** 그대로 보입니다. 실패한 실행은 빨간색.

- 실패한 실행 → **Debug in editor** (성공한 실행은 **Copy to editor**) → 그때의 데이터가 편집 화면으로 불려와 첫 노드에 고정됨 → 고치면서 다시 실험

Pin data = 테스트용 데이터 고정 노드 OUTPUT 의 📌 아이콘. 그 노드의 결과를 고정해서, 다음 실행부터는 **진짜로 실행하지 않고 고정된 데이터를 씀**. → 메일을 매번 진짜로 보내거나, 폼을 매번 다시 제출하거나, 유료 API 를 계속 부르지 않고 뒤쪽만 고칠 수 있습니다.

노드별 실패 대처 (노드 Settings 탭)

설정	뜻
Retry On Fail	실패하면 N번 다시 시도 (일시적 오류 대비)
On Error → Stop Workflow	기본값. 거기서 멈춤
On Error → Continue	무시하고 다음 노드로
On Error → Continue (using error output)	에러 전용 출구로 보내서 따로 처리

Error Workflow = 사고 신고 센터 "어떤 워크플로우든 실패하면 나에게 텔레그램으로 알려줘" 를 만드는 법:

1. 새 워크플로우 `00-에러 알림` → **Error Trigger** → Telegram Send Message
   - Text: `🚨 {{ $json.workflow.name }} 실패\n노드: {{ $json.execution.lastNodeExecuted }}\n{{ $json.execution.error.message }}`
2. 감시할 워크플로우 → 오른쪽 위 **⋯ (Options) → Settings → Error workflow** 에서 `00-에러 알림` 선택 → **Save**
3. Error Trigger 가 든 워크플로우는 **Publish 하지 않아도** 작동합니다

> ⚠️ Error Workflow 는 **자동 실행(Publish 된 트리거)** 이 실패할 때 동작합니다. 내가 편집 화면에서 직접 누른 테스트 실행은 알리지 않습니다.

❓ 왜 필요해?

자동화는 **내가 안 볼 때** 돕니다. 그래서 실패해도 **내가 모릅니다.** 매일 아침 알림이 일주일째 안 오고 있었는데 아무도 몰랐다 — 가장 흔한 사고입니다.

⚠️ 자주 하는 실수

- Pin 해둔 걸 잊고 "왜 데이터가 안 바뀌지?" → 📌 가 켜진 노드 확인, 다 만들면 **Unpin**
- Continue 로 에러를 숨김 → 조용히 틀린 결과가 쌓임. **에러 출구로 받아서 기록**하기
- Error Workflow 를 만들기만 하고 각 워크플로우 Settings 에 **연결 안 함**

💬 AI 에게 이렇게 말하기

```
n8n 실행 기록에 이 에러가 떴어. 무슨 뜻인지, 어느 노드 설정을 확인해야 하는지 알려줘.
(에러 메시지와 노드 이름 붙여넣기)
```

13단계

진짜로 켜기와 관리 (Publish · 버전 · 내보내기)

📖 이게 뭐야?

초안(Draft) vs 게시(Published)

```
편집 화면에서 고친 것 ─자동 저장─▶ [ 초안 ]  ─Publish─▶ [ 게시 버전 ] ← 실제로 돌아가는 것
```

게시 후에 고쳐도 **실제 운영은 그대로**입니다. 다시 Publish 해야 바뀝니다. 그래서 운영 중인 자동화를 **안심하고** 고칠 수 있습니다.

Publish 버튼 상태

표시	뜻
Publish (회색)	아직 한 번도 안 켬 / 바뀐 게 없음
Published	켜져서 돌고 있음
Published, has changes	고친 게 있는데 아직 반영 안 함
Failed to publish	트리거 설정 문제로 못 켬 (열쇠·주소 확인)

버전 기록 (History) 위쪽 ⏱ 아이콘 → 예전 버전 보기·되돌리기. Publish 할 때 **버전 이름**을 남겨두면 찾기 쉽습니다 (`v1 - 첫 운영`).

내보내기 / 가져오기 (Export · Import)

- 워크플로우 ⋯ → **Download** → `.json` 파일 (열쇠는 빠지고 구조만)
- 가져올 때: 새 워크플로우에서 ⋯ → **Import from File**, 또는 JSON 을 복사해서 **캔버스에 Ctrl+V**
- → 클로드가 만든 워크플로우 JSON 도 이렇게 붙여넣을 수 있습니다 (14단계)

Templates = 남이 만든 레시피 n8n.io/workflows 에 수천 개의 공개 레시피. 왼쪽 메뉴 **Templates** 로 열 수 있고, 템플릿 페이지에서 사용 버튼 → 내 n8n 으로 복사 → 열쇠만 연결하면 작동.

🛠️ 직접 해보기

1. 7단계 신청서 워크플로우 → Edit Fields 문구 수정 → 버튼이 **Published, has changes** 로 바뀌는지 확인
2. Production URL 로 제출 → **예전 문구**로 처리되는지 Executions 에서 확인
3. **Shift + P** → 버전 이름 `v2 - 문구 수정` → Publish → 다시 제출 → 새 문구 ✅
4. ⋯ → **Download** → 파일 저장 (체험 끝나기 전 백업 습관)

❓ 왜 필요해?

수업용 Cloud 는 14일 뒤 사라집니다. 그리고 운영 중인 자동화를 실수로 망가뜨리지 않으려면 **초안/게시 구분**과 **백업**이 필수입니다.

⚠️ 자주 하는 실수

- 고치고 Publish 안 함 → 계속 예전 버전이 돎
- JSON 을 받은 쪽에서 "왜 안 돼?" → **열쇠(Credential)는 다시 연결**해야 함
- 체험 종료 후 백업하려고 함 → 워크스페이스는 삭제됨 (다운로드는 90일 안에만)

💬 AI 에게 이렇게 말하기

```
이 n8n 워크플로우 JSON 을 읽고, 어떤 트리거로 시작해서 각 노드가 무슨 일을 하는지
초보자용으로 단계별 설명해줘. 그리고 Publish 전에 확인할 점 3가지도.
(JSON 붙여넣기)
```

14단계

AI 로 워크플로우 만들기 (n8n Assistant · 클로드)

📖 이게 뭐야?

n8n Assistant (Ctrl/Cmd + J) n8n 안에 들어 있는 AI 도우미입니다. "이 에러 왜 나?", "이 노드 어떻게 설정해?" 를 물을 수 있고, **"매일 아침 날씨를 텔레그램으로 보내는 워크플로우 만들어줘"** 처럼 말하면 **워크플로우를 직접 만들어** 줍니다 (AI Workflow Builder). Cloud 체험에는 Assistant 크레딧이 포함되어 있습니다 (요청 1번 = 1크레딧).

클로드로 만들기 클로드에게 **n8n 워크플로우 JSON** 을 만들어 달라고 한 뒤 → 캔버스에 **Ctrl+V**. 열쇠만 연결하면 됩니다.

❓ 왜 필요해?

기본기(2~13단계)를 알면 AI 가 만든 워크플로우를 **읽고 고칠 수** 있습니다. 모르면 AI 가 만든 걸 그냥 믿어야 합니다 — 바이브 코딩과 똑같습니다.

🛠️ 좋은 요청 = 세 칸 설계도

'한눈에 보기' 의 "① 언제 ② 무엇을 ③ 어디로" 를 그대로 말하면 됩니다.

```
① 언제: 평일 오전 8시
② 무엇을: Open-Meteo 에서 서울 날씨(최고/최저/비 올 확률)를 가져와서 Gemini 로 옷차림 한 줄 추천
③ 어디로: 내 텔레그램 봇으로 전송
조건: 비 올 확률 60% 이상이면 메시지 맨 앞에 ☔ 붙이기
```

⚠️ 자주 하는 실수

- "자동화 만들어줘" 처럼 막연하게 → AI 가 아무 서비스나 골라 엉뚱한 걸 만듦
- AI 가 만든 걸 **Execute 없이 바로 Publish** → 노드마다 Execute step 으로 확인 먼저
- 클로드가 만든 JSON 에 **예전 버전 노드**가 섞임 → 노란 경고가 뜨는 노드는 새로 추가해서 교체

💬 AI 에게 이렇게 말하기

n8n Assistant 에게

```
평일 오전 8시에 Open-Meteo 로 서울 날씨를 가져와서, Gemini 로 옷차림을 한 줄 추천하고,
텔레그램으로 보내는 워크플로우를 만들어줘. 비 올 확률 60% 이상이면 맨 앞에 ☔ 를 붙여줘.
```

클로드에게

```
n8n 2.x 에 붙여넣을 수 있는 워크플로우 JSON 을 만들어줘. (Ctrl+V 로 캔버스에 붙일 거야)
① 언제: Form Trigger (이름, 이메일, 문의 내용)
② 무엇을: If 로 문의 내용에 "환불" 포함 여부 확인
③ 어디로: 포함이면 텔레그램 알림, 아니면 Gmail 자동 답장
Credential 은 비워두고, 노드 이름은 한글로 하는 일을 적어줘. 각 노드가 하는 일도 표로 설명해줘.
```

부록 A

노드 치트시트

하고 싶은 것	노드
내가 눌러서 시작	Manual Trigger
정해진 시간마다	Schedule Trigger
웹 신청서	Form Trigger (On form submission)
채팅으로 시작	Chat Trigger (On chat message)
다른 앱이 호출	Webhook
필드 만들기·정리	Edit Fields (Set)
조건으로 거르기	Filter
두 갈래 / 여러 갈래	If / Switch
합치기	Merge
목록 쪼개기 / 모으기	Split Out / Aggregate
정렬 / 개수 제한 / 중복 제거	Sort / Limit / Remove Duplicates

하고 싶은 것	노드
나눠서 반복	Loop Over Items
잠깐 멈추기	Wait
아무 API 호출	HTTP Request
직접 코드	Code
n8n 안의 표	Data table
AI 직원	AI Agent (+ Chat Model, Memory, Tool)
AI 한 번 처리	Basic LLM Chain / Gemini·OpenAI·Anthropic 노드
실패 알림	Error Trigger
메일 / 시트 / 텔레그램	Gmail / Google Sheets / Telegram

부록 B

에러 메시지 번역표

화면에 뜬 말	쉬운 말	해결
`No output data returned` / 빈 OUTPUT	앞에서 item 이 0개 왔음	앞 노드 OUTPUT 확인, Filter/If 조건 확인
`Referenced node doesn't exist`	`$('이름')` 의 노드가 없음	노드 이름 바뀌었는지 확인
`Can't get data for expression`	그 노드가 아직 실행 안 됨	앞 노드부터 Execute
표현식 칸이 빨간색	공식 문법 오류	`{{ }}` 짝, 따옴표, 필드 이름 확인
`Credentials not found` / `401 Unauthorized`	열쇠 없음·틀림	Credential 다시 연결
`403 Forbidden` / `insufficient permission`	권한 부족	Google 재연결 시 권한 체크 모두 허용
`404 Not Found`	주소·ID 틀림	URL, 시트 ID, Chat ID 확인
`429 Too Many Requests` / quota	너무 자주 호출	Wait, Loop Over Items, 잠시 후
`chat not found` (Telegram)	봇이 그 사람을 모름	봇에게 먼저 말 걸기, Chat ID 확인
`The workflow has issues`	설정 안 된 칸이 있음	⚠️ 표시된 노드 열어 빈칸 채우기
Production URL 이 `404` / 응답 없음	게시 안 됨	**Publish** 확인

화면에 뜬 말	쉬운 말	해결
Test URL 이 응답 없음	듣고 있지 않음	**Execute step / Listen for test event** 누른 뒤 요청
`Workflow execution timed out`	너무 오래 걸림 (체험판 180초)	나눠서 처리 (11단계)

부록 C

자동화 설계 10계명

1. 노드를 놓기 전에 ① **언제** ② **무엇을** ③ **어디로** 를 종이에
2. **노드 하나 놓고 → Execute step → OUTPUT 확인**
3. OUTPUT 위의 **item 개수**를 항상 본다
4. 노드 이름 = **하는 일** (`신청자 정리` , `합격 메일`)
5. 비밀번호·키는 **Credential 에만**
6. 테스트는 **나에게** 먼저 보낸다 (메일 받는 사람을 나로)
7. 반복 테스트는 **Pin data** 로, 끝나면 Unpin
8. AI 에겐 **System Message** 와 **읽기 도구**부터
9. **Publish** 해야 진짜로 돈다. 고쳤으면 **다시 Publish**
10. **Error Workflow** 를 연결하고, 주기적으로 **Download** 백업